

EXPLICACAO ORDENADA DO CODIGO - APP CARDAPIO

RESUMO DA LOGICA

O app tem dois modos. Com internet, ele inicia um servidor mock HTTP dentro do proprio app, busca um JSON por HTTP, baixa as imagens desse mock, salva os dados no SQLite e salva as fotos no sistema de arquivos. Sem internet, ele le o SQLite e mostra as fotos salvas. No offline, o preco vira "a consultar".

ARQUIVOS PRINCIPAIS

1. MainActivity.kt: tela principal, fluxo online/offline, leitura JSON, download das imagens e montagem dos cards.
2. MockCardapioServer.kt: servidor HTTP mock interno que entrega o JSON e as imagens.
3. CardapioDBHelper.kt: banco SQLite.
4. activity_main.xml: tela base com ScrollView e LinearLayout.
5. AndroidManifest.xml: permissoes de internet e estado da rede.
6. assets/foto1.jpg a foto4.jpg: imagens usadas pelo mock.

MAINACTIVITY.KT

package com.example.trab_mobile_m2
Define o pacote da classe.

imports

Trazem classes prontas do Android e Java, como TextView, ImageView, HttpURLConnection, JSONArray, File e SQLite.

class CardapioItem

Classe simples para representar um prato.

var id: Int = 0

Guarda o id do prato.

var nome: String = ""

Guarda o nome do prato.

var preco: Double = 0.0

Guarda o preco com casas decimais.

var imagemLocal: String = ""

Guarda o caminho da imagem salva no armazenamento interno.

class MainActivity : AppCompatActivity()

Define a tela principal. O sinal ":" indica heranca.

private lateinit var containerCardapio: LinearLayout

Variavel que recebe o LinearLayout do XML. lateinit significa que sera inicializada depois.

private lateinit var dbHelper: CardapioDBHelper

Objeto usado para acessar o SQLite.

private var mockServer: MockCardapioServer? = null

Objeto do servidor mock. O "?" indica que pode ser nulo.

private val mainHandler = Handler(Looper.getMainLooper())

Permite voltar para a thread principal antes de mexer na interface.

```
private lateinit var cardapioUrl: String
```

Guarda a URL do JSON gerada pelo servidor mock.

```
onCreate
```

Funcao chamada quando a tela abre.

```
super.onCreate(savedInstanceState)
```

Executa a inicializacao padrao da Activity.

```
setContentView(R.layout.activity_main)
```

Liga o codigo Kotlin ao layout XML.

```
containerCardapio = findViewById(R.id.container_cardapio)
```

Busca o LinearLayout do XML.

```
dbHelper = CardapioDBHelper(this)
```

Cria o helper do SQLite.

```
if (temConexaoInternet())
```

Verifica se existe internet para decidir entre online e offline.

```
mockServer = MockCardapioServer(this)
```

Cria o servidor mock interno.

```
mockServer!!.start()
```

Inicia o servidor mock. O "!!" indica que o objeto nao deve ser nulo nesse momento.

```
cardapioUrl = mockServer!!.getCardapioUrl()
```

Pega a URL HTTP do JSON mock.

```
carregarDadosOnline()
```

Busca os dados por HTTP.

```
else carregarDadosOffline()
```

Se nao houver internet, carrega os dados salvos localmente.

```
onDestroy
```

Funcao chamada quando a tela e destruida.

```
mockServer?.stop()
```

Para o servidor mock se ele existir. O "?" so chama se nao for nulo.

```
temConexaoInternet
```

Verifica se o Android possui uma rede ativa com internet.

```
getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
```

Pega o servico de conectividade do Android.

```
connectivityManager.activeNetwork
```

Pega a rede ativa.

```
if (network == null) return false
```

Se nao existe rede, retorna falso.

getNetworkCapabilities(network)

Pega as capacidades da rede.

hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)

Retorna verdadeiro se a rede tem internet.

carregarDadosOnline

Funcao responsavel pelo fluxo online.

Thread { ... }.start()

Cria uma thread separada porque HTTP nao deve rodar na thread principal.

try

Tenta executar a carga online.

val jsonString = baixarTexto(cardapioUrl)

Baixa o JSON como texto.

val jsonArray = JSONArray(jsonString)

Transforma o texto JSON em uma lista JSON.

val itens = ArrayList<CardapioItem>()

Cria uma lista vazia de pratos.

for (i in 0 until jsonArray.length())

Percorre todos os itens do JSON.

val itemJson = jsonArray.getJSONObject(i)

Pega o objeto JSON da posicao atual.

val item = CardapioItem()

Cria um prato vazio.

item.id = itemJson.getInt("id")

Le o id do JSON.

item.nome = itemJson.getString("nome")

Le o nome do JSON.

item.preco = itemJson.getDouble("preco")

Le o preco do JSON.

val urlImagem = itemJson.getString("url_imagem")

Le a URL da imagem no JSON.

item.imagemLocal = baixarESalvarImagem(...)

Baixa a imagem por HTTP e guarda o caminho local.

itens.add(item)

Adiciona o prato na lista.

salvarCargaLocal(itens)

Salva a lista no SQLite.

mainHandler.post { ... }

Volta para a thread principal para atualizar a tela.

exibirCardapioNaTela(itens, true)

Mostra os cards com preco, porque esta online.

catch (e: Exception)

Se der erro no online, executa o bloco de erro.

carregarDadosOffline()

Plano B: carrega dados locais.

baixarTexto(urlString)

Funcao que baixa texto de uma URL.

URL(urlString)

Transforma texto em URL.

openConnection() as HttpURLConnection

Abre uma conexao HTTP.

requestMethod = "GET"

Define que a requisicao e para buscar dados.

connectTimeout e readTimeout

Definem o tempo maximo de conexao e leitura.

responseCode

Codigo HTTP da resposta.

if (codigoResposta < 200 || codigoResposta > 299)

Se nao for sucesso HTTP, lanca erro.

BufferedReader(InputStreamReader(inputStream))

Prepara a leitura do texto recebido.

reader.readText()

Le todo o JSON.

reader.close() e inputStream.close()

Fecham os recursos.

finally { conexao.disconnect() }

Fecha a conexao mesmo com erro.

baixarESalvarImagem

Baixa uma imagem por HTTP e salva no armazenamento interno.

BitmapFactory.decodeStream(inputStream)

Transforma os bytes da imagem em Bitmap.

if (bitmap == null)

Se a imagem não for válida, lança erro.

`openFileOutput(nomeArquivo, Context.MODE_PRIVATE)`

Cria arquivo privado do app.

`bitmap.compress(Bitmap.CompressFormat.JPEG, 90, outputStream)`

Salva a imagem em JPEG com qualidade 90.

`File(filesDir, nomeArquivo).absolutePath`

Retorna o caminho completo da imagem salva.

`salvarCargaLocal`

Salva os pratos no SQLite.

`dbHelper.limparBanco()`

Apaga dados antigos para não duplicar.

`for (i in 0 until itens.size)`

Percorre a lista de pratos.

`dbHelper.salvarPrato(...)`

Salva cada prato no banco.

`carregarDadosOffline`

Le os pratos salvos no SQLite.

`dbHelper.readableDatabase`

Abre o banco para leitura.

`val sql = "SELECT ..."`

Monta a consulta SQL.

`db.rawQuery(sql, null)`

Executa a consulta e retorna um Cursor.

`getColumnIndexOrThrow`

Descobre a posição de cada coluna.

`while (cursor.moveToNext())`

Percorre as linhas do resultado.

`cursor.getInt, getString, getDouble`

Le os valores do banco.

`itens.add(item)`

Adiciona o item local na lista.

`cursor.close()` e `db.close()`

Fecham cursor e banco.

`exibirCardapioNaTela(itens, false)`

Mostra os dados em modo offline.

`exibirCardapioNaTela`

Monta a interface.

`containerCardapio.removeAllViews()`

Limpa a tela antes de desenhar novamente.

`setBackgroundColor` e `setPadding`

Define cor de fundo e espacamento.

`adicionarCabecalho(online)`

Adiciona titulo e status.

`if (lista.isEmpty())`

Se nao houver dados locais, mostra mensagem.

`for (i in 0 until lista.size)`

Percorre os itens.

`criarCardItem(item, online)`

Cria o card de cada prato.

`adicionarCabecalho`

Cria o titulo e o texto de status.

`TextView(this)`

Cria texto por codigo.

`text, textSize, typeface, setTextColor, gravity`

Configuram conteudo, tamanho, negrito, cor e alinhamento.

`adicionarMensagemVazia`

Mostra aviso se nao ha cache local.

`criarCardItem`

Cria o card visual com imagem, nome e preco.

`LinearLayout(this)`

Cria o card.

`card.orientation = LinearLayout.VERTICAL`

Organiza imagem, nome e preco um embaixo do outro.

`GradientDrawable`

Cria fundo branco com borda.

`ImageView(this)`

Cria o componente de imagem.

`imagem.scaleType = CENTER_CROP`

Faz a imagem preencher o espaco.

`File(item.imagemLocal)`

Aponta para a imagem salva.

`if (arquivoFoto.exists())`

Verifica se a imagem existe.

BitmapFactory.decodeFile

Le a imagem salva.

imagem.setImageBitmap(bitmap)

Mostra a foto no card.

nome.text = item.nome

Mostra o nome do prato.

if (online)

Decide se mostra preco real ou "a consultar".

String.format(... "R\$ %.2f", item.preco)

Formata o preco com duas casas.

else preco.text = "a consultar"

Atende ao requisito offline.

return card

Devolve o card pronto.

dp(value)

Converte valores para dp, unidade visual do Android.

MOCKCARDAPIOSERVER.KT

class MockCardapioServer(private val context: Context)

Classe que cria o servidor mock. Recebe Context para acessar assets.

serverSocket

Objeto que escuta requisicoes HTTP.

running

Controla se o servidor esta rodando.

port

Guarda a porta escolhida automaticamente.

start()

Inicia o servidor.

ServerSocket(0, 50, localAddress)

Cria servidor em 127.0.0.1. O 0 escolhe uma porta livre.

Thread { while (running) ... }.start()

Deixa o servidor aguardando requisicoes em segundo plano.

accept()

Espera um cliente conectar.

atenderCliente(client)

Atende cada requisicao recebida.

stop()

Para o servidor.

getCardapioUrl()

Retorna a URL do JSON mock.

input.read(request)

Le a requisicao HTTP.

firstLine.split(" ")

Separa metodo, caminho e versao HTTP.

if (path == "/cardapio.json")

Se pediu o JSON, devolve o cardapio.

else if (path == "/foto1.jpg")

Se pediu uma imagem, devolve o asset correspondente.

montarJsonCardapio()

Monta o JSON com 4 registros.

baseUrl

Usado para montar as URLs das imagens no proprio servidor mock.

enviarAsset

Le uma imagem da pasta assets e devolve por HTTP.

context.assets.open(assetName)

Abre o arquivo dentro de assets.

ByteArrayOutputStream

Guarda os bytes da imagem antes de responder.

enviarResposta

Monta a resposta HTTP com status, tipo e conteudo.

Content-Type

Informa se a resposta e JSON, texto ou imagem.

Content-Length

Informa o tamanho da resposta.

CARDAPIODBHELPER.KT

class CardapioDBHelper : SQLiteOpenHelper

Classe que gerencia o SQLite.

DATABASE_NAME

Nome do banco.

DATABASE_VERSION

Versao do banco.

TABLE_NAME

Nome da tabela.

COLUMN_ID, COLUMN_NOME, COLUMN_PRECO, COLUMN_IMAGEM_LOCAL

Nomes das colunas.

onCreate(db)

Cria a tabela na primeira execucao.

CREATE TABLE

Comando SQL que cria a tabela pratos.

INTEGER PRIMARY KEY

Define id como chave primaria.

TEXT

Tipo usado para nome e caminho da imagem.

REAL

Tipo usado para preco.

onUpgrade

Executa quando a versao do banco muda.

DROP TABLE IF EXISTS

Apaga tabela antiga.

salvarPrato

Salva ou atualiza um prato.

ContentValues

Estrutura usada para enviar valores ao SQLite.

cv.put

Coloca valor em cada coluna.

insertWithOnConflict(... CONFLICT_REPLACE)

Insere ou substitui se ja existir o mesmo id.

limparBanco

Apaga registros antigos.

execSQL("DELETE FROM pratos")

Executa comando SQL que limpa a tabela.

ANDROIDMANIFEST.XML

INTERNET

Permite requisicao HTTP.

ACCESS_NETWORK_STATE

Permite verificar se ha internet.

usesCleartextTraffic="true"

Permite HTTP sem HTTPS, necessario para o mock local.

MainActivity exported true
Define a tela inicial do app.

ACTIVITY_MAIN.XML

ScrollView
Permite rolar a tela.

LinearLayout container_cardapio
Espaco onde os cards sao adicionados por codigo.

COMO DEMONSTRAR

1. Rodar o app pelo Android Studio com internet ligada.
2. Mostrar que aparecem 4 itens com imagem, nome e preco.
3. Explicar que os dados vieram de um JSON servido por mock HTTP interno.
4. Fechar o app.
5. Desligar internet do emulador.
6. Abrir o app de novo.
7. Mostrar que os dados continuam aparecendo pelo SQLite.
8. Mostrar que o preco virou "a consultar".

FRASE FINAL

Online, o app consome um JSON por HTTP vindo de um mock interno, baixa as imagens e salva tudo localmente. Offline, ele consulta o SQLite e usa as imagens salvas no armazenamento interno, exibindo "a consultar" no lugar do preco.